API Reference Manual For Tele

AICXTEK Confidential

Documentation Serial Number:	
Documentation Classification:	**Confidential**
Documentation Owner:	**AICXTEK**
Documentation Version:	**1.0**

Author:	**R&D Center**
Date:	**2023.6**

版本更新记录			
版本号	日期	作者	版本更新描述
1.0	2023-06-28	R&D Center	文档建立
1.0.1	2023-07-31	R&D Center	统一tele service规则
1.0.2	2023-11-03	R&D Center	补充新增接口
1.0.3	2024-01-05	R&D Center	更新文档

AICXTEK Confidential

本文档（包括任何附件，及其不时的修订）为归芯科技（深圳）有限公司及其关联公司（合称“归芯”）的保密信息。归芯有权不时修订本文档。本文档如有变更，恕不另行通知。
除非已另行获得归芯的明确授权，请勿向第三方透露本文档的任何内容，请勿将本文档的任何内容以任何方式进行复制、发布、修改、传播、分发、翻印、拆解、反向工程、反编译或者为任何目的使用。
归芯始终持有本文档的全部所有权和著作权、专利权、商业秘密、商标权和其他知识产权。除非归芯另行明确授予有关权利，本文档不提供有关上述知识产权的任何权利。
归芯按“现状”提供本文档及归芯产品或服务。归芯未对本文档中的任何内容或本文档涉及的归芯的产品或服务作出任何明示或暗示的陈述、保证或承诺，包括但不限于任何针对特定目的的适销性或适用性，准确性或完整性，或不侵权。
在法律允许的范围内，归芯不对与本文档有关或因本文档内容造成的任何损失承担责任。在任何情况下，归芯都不对因使用本文档造成的或与之有关的任何间接、偶然的、附带或惩罚性损失，或任何利润、收入的损失承担责任。

This document (including any attachment, as amended from time to time) is confidential information of AICXTEK technology co.,ltd. and its affiliates (collectively referred to as “AICXTEK”). AICXTEK reserves the right to amend this document from time to time. Any changes to this document will not be notified.
Unless otherwise expressly authorized by AICXTEK, you should not disclose any of this document to any third parties. Nor should you copy, publish, modify, broadcast, distribute, photocopy, disassemble, reverse engineer, or decompile any of this document or use it for any purpose.
AICXTEK retains all rights of ownership and copyright, patent rights, trade secrets, trademark rights and other intellectual property rights pertaining to this document. Except for rights otherwise expressly granted by AICXTEK, this document does not provide any rights to any of the above intellectual property rights.
AICXTEK provides this document and AICXTEK’s products or services on a “as-is” basis. AICXTEK does not provide any representations or warranties on any of this document, or AICXTEK’s products or services, whether express or implied, including without limitation, merchantability, fitness for a particular purpose, accuracy or completeness, or non-infringement.
To the extent permitted by law, AICXTEK is not liable for any loss relating to or arising from this document. AICXTEK is not in any event be liable for any indirect, incidental or punitive damages, or loss of profits or revenue.

AICXTEK Confidential

目录

1. 目的 3
2. 适用范围 3
3. 主要内容 3
3.1. Call 3
3.1.1. aic_srv_tele_call_register 3
3.1.2. aic_srv_tele_call_unregister 7
3.1.3. aic_srv_tele_call_dial 7
3.1.4. aic_srv_tele_call_answer 8
3.1.5. aic_srv_tele_call_release 8
3.1.6. aic_srv_tele_call_reject 8
3.1.7. aic_srv_tele_call_volte_enable 9
3.1.8. aic_srv_tele_call_volte_is_enable 9
3.1.9. aic_srv_tele_call_is_emergency_number 9
3.1.10. aic_srv_tele_call_send_dtmf 10
3.1.11. aic_srv_tele_call_have_volte_reg 10
3.1.12. aic_srv_tele_call_set_auto_answer 10
3.1.13. aic_srv_tele_call_get_auto_answer 11
3.1.14. aic_srv_tele_call_get_call_info_list 11
3.2. Radio 12
3.2.1. aic_srv_tele_radio_register 12
3.2.2. aic_srv_tele_radio_unregister 13
3.2.3. aic_srv_tele_radio_get_singnal_info 13
3.2.4. aic_srv_tele_radio_get_operator_info 18
3.2.5. aic_srv_tele_radio_on 19
3.2.6. aic_srv_tele_radio_is_on 19
3.2.7. aic_srv_tele_radio_get_reg_status 19
3.2.8. aic_srv_tele_radio_have_ims_reg 20
3.2.9. aic_srv_tele_radio_set_imei 20
3.2.10. aic_srv_tele_radio_get_imei 20
3.2.11. aic_srv_tele_radio_have_cali 21
3.2.12. aic_srv_tele_radio_get_cell_info 21
3.2.13. aic_srv_tele_radio_get_neighbor_cell_info_list 21
3.2.14. aic_srv_tele_radio_set_filter 22
3.2.15. aic_srv_tele_radio_start_search 22
3.3. SIM 23
3.3.1. aic_srv_tele_sim_register 23
3.3.2. aic_srv_tele_sim_unregister 24
3.3.3. aic_srv_tele_sim_get_status 24
3.3.4. aic_srv_tele_sim_set_pin_lock_status 25
3.3.5. aic_srv_tele_sim_get_pin_lock_status 26

AICXTEK Confidential

3.3.6. aic_srv_tele_sim_unlock_pin....26
3.3.7. aic_srv_tele_sim_change_pin....27
3.3.8. aic_srv_tele_sim_get_pin_remain_times....28
3.3.9. aic_srv_tele_sim_select_card....28
3.3.10. aic_ srv_tele_sim_get_current_card....28
3.3.11. aic_ srv_tele_sim_get_exist_status....29
3.3.12. aic_srv_tele_sim_get_imsi....29
3.3.13. aic_ srv_tele_sim_get_msisdn....29
3.4. Data....29
3.4.1. aic_srv_tele_data_register....30
3.4.2. aic_srv_tele_data_unregister....30
3.4.3. aic_srv_tele_data_get_internet_pdp_info....31
3.5. SMS....31
3.5.1. aic_srv_tele_sms_register....31
3.5.2. aic_srv_tele_sms_unregister....32
3.5.3. aic_srv_tele_sms_send....32
3.5.4. aic_srv_tele_sms_del_in_sim....33
3.5.5. aic_srv_tele_sms_set_smsc....34
3.5.6. aic_srv_tele_sms_get_smsc....35
3.5.7. aic_srv_tele_sms_get_list_in_sim....35
3.5.8. aic_srv_tele_sms_read_in_sim....36
3.5.9. aic_srv_tele_sms_enable_status_report....37
3.5.10. aic_srv_tele_sms_set_valid_period....37
3.6. Phonebook....37
3.6.1. aic_srv_tele_pb_register....38
3.6.2. aic_srv_tele_pb_unregister....38
3.6.3. aic_srv_tele_pb_get_status....38
3.6.4. aic_srv_tele_pb_write....39
3.6.5. aic_srv_tele_pb_read....39

AICXTEK Confidential

1. 目的

本文档包含 tele service 各模块的功能范围、接口列表以及事件上报等信息的描述，旨在为用户提供 tele service 的功能支持。

2. 适用范围

本制度适用于归芯科技（深圳）有限公司及其分、子公司研发中心员工。

3. 主要内容

3.1. Call

简要描述

Tele Call Service模块主要提供了与Call相关的功能接口，包括基础的电话业务、呼叫转移、呼叫等待、呼叫限制等补充业务以及USSD等相关功能。

Header file	Location
aic_srv_tele_call.h	

3.1.1. aic_srv_tele_call_register

Prototype	int32_t aic_srv_tele_call_register(aic_srv_bus_info_t *p_bus_info, srv_func_callback _srv_callback);
Function	tele call service的注册接口，注册的callback用于接收service层上报的事件
Constraint	typedef struct { /* client的名称 */ char *p_client_name; /* TODO: client关注的信号相关数据信息 */ }aic_srv_bus_info_t; typedef int32_t (*srv_func_callback)(void *p_param, uint32_t size);
Parameters	@p_bus_info: [In]需要client传给service层当前client的名称以及client需要关注的信号相关数据信息 @_srv_callback: [In]client传给service层的回调函数 该回调函数中的参数p_param上报数据结构体指针，参数size表示数据的大小（包括event id）

AICXTEK Confidential

AICXTEK Confidential

目前可已报的事件有：

AICXTEK Confidential

	件），携带ts_call_status_updated_t数据信息； 其中，p_param可强转为ts_event_call_t指针，对应的事件id是TS_EVENT_CALL_STATUS_UPDATED; size值为sizeof(ts_call_status_updated_t)； 2、 TS_EVENT_CALL_VOLTE_REG_STATUS_UPDATED（VoLTE注册状态发生变化的事件），携带ts_call_volte_reg_status_updated_t数据信息； 其中，p_param可强转为ts_event_call_t指针，对应的事件id是TS_EVENT_CALL_VOLTE_REG_STATUS_UPDATED; size值为sizeof(ts_call_volte_reg_status_updated_t)； 3、 TS_EVENT_CALL_RINGBACK_VOICE（回铃音事件），携带ts_call_ringback_voice_t数据信息. 其中，p_param可强转为ts_event_call_t指针，对应的事件id是TS_EVENT_CALL_RINGBACK_VOICE; size值为sizeof(ts_call_ringback_voice_t). 4、 TS_EVENT_CALL_DIAL_RESP（调用拨号接口后的响应事件），携带ts_call_dial_resp_t数据信息（拨号的返回值）. 5、 TS_EVENT_CALL_ANSWER_RESP（调用接听接口后的响应事件），携带ts_call_answer_resp_t数据信息（接听的返回值）. 6、 TS_EVENT_CALL_RELEASE_RESP（调用挂断接口后的响应事件），携带ts_call_release_resp_t数据信息（挂断的返回值）. 7、 TS_EVENT_CALL_REJECT_RESP（调用拒接接口后的响应事件），携带ts_call_reject_resp_t数据信息（拒接的返回值）. 8、 TS_EVENT_CALL_RECEIVE_DTMF（接收到DTMF的事件），携带ts_call_receive_dtmf_t数据信息. 9、 TS_EVENT_CALL_SEND_DTMF_RESP（调用发送DTMF接口后的响应事件），携带ts_call_send_dtmf_resp_t数据信息. 10、 TS_EVENT_CALL_VOLTE_ENABLE_RESP（调用使能VoLTE接口后的响应事件），携带ts_call_volte_enable_resp_t数据信息. TODO: 后续在这里补充callback中返回event id的列表.
Return value	返回给client对应的client handle，其中0代表出错

AICXTEK Confidential

3.1.2. aic_srv_tele_call_unregister

Prototype	int32_t aic_srv_tele_call_unregister(int32_t client_handle);
Function	tele call service的去注册接口
Constraint	
Parameters	@client_handle: [In]调用aic_srv_tele_call_register接口时返回给client的client handle值
Return value	成功返回0，失败返回其它值

3.1.3. aic_srv_tele_call_dial

Prototype	int32_t aic_srv_tele_call_dial(ts_call_dial_info_t *p_dial_info, srv_func_callback _response_callback);
Function	拨打电话接口
Constraint	/* 电话号码的最大长度 */ #define TS_CALL_PHONE_NUMBER_LEN_MAX (32) typedef struct { /* sim卡的id，0-卡槽1，1-卡槽2 */ uint8_t card_id; /* 拨打的电话号码 */ char phone_number[TS_CALL_PHONE_NUMBER_LEN_MAX + 1]; /* 主叫线路识别限制 */ int32_t clir; }ts_call_dial_info_t;
Parameters	@p_dial_info: [In]拨打电话的信息 @_response_callback: [In] client调用该接口时传入的响应回调函数，在at命令的response结束后被调用. 响应的事件id是TS_EVENT_CALL_DIAL_RESP，携带ts_call_dial_resp_t数据信息. 其中，p_param可强转为ts_event_call_t指针，对应的事件id是TS_EVENT_CALL_DIAL_RESP； size值为sizeof(ts_call_dial_resp_t).
Return value	成功返回0，失败返回其它值

AICXTEK Confidential

3.1.4. aic_srv_tele_call_answer

Prototype	int32_t aic_srv_tele_call_answer(srv_func_callback _response_callback);
Function	接听来电接口
Constraint	无
Parameters	@_response_callback: [In] client调用该接口时传入的响应回调函数，在at命令的response结束后被调用. 响应的事件id是TS_EVENT_CALL_ANSWER_RESP，携带ts_call_answer_resp_t数据信息. 其中，p_param可强转为ts_event_call_t指针，对应的事件id是TS_EVENT_CALL_ANSWER_RESP; size值为sizeof(ts_call_answer_resp_t).
Return value	成功返回0，失败返回其它值

3.1.5. aic_srv_tele_call_release

Prototype	int32_t aic_srv_tele_call_release(uint8_t card_id, uint8_t call_idx, srv_func_callback _response_callback);
Function	挂断call_idx对应的电话接口
Constraint	
Parameters	@card_id: [In]sim卡的id，0-卡槽1，1-卡槽2 @call_idx: [In]要挂断的电话index值 @_response_callback: [In] client调用该接口时传入的响应回调函数，在at命令的response结束后被调用. 响应的事件id是TS_EVENT_CALL_RELEASE_RESP，携带ts_call_release_resp_t数据信息. 其中，p_param可强转为ts_event_call_t指针，对应的事件id是TS_EVENT_CALL_RELEASE_RESP; size值为sizeof(ts_call_release_resp_t).
Return value	成功返回0，失败返回其它值

3.1.6. aic_srv_tele_call_reject

Prototype	int32_t aic_srv_tele_call_reject(srv_func_callback _response_callback);
Function	拒接来电的接口
Constraint	

AICXTEK Confidential

Parameters	@_response_callback: [In] client调用该接口时传入的响应回调函数，在at命令的response结束后被调用. 响应的事件id是TS_EVENT_CALL_REJECT_RESP，携带ts_call_reject_resp_t数据信息. 其中，p_param可强转为ts_event_call_t指针，对应的事件id是TS_EVENT_CALL_REJECT_RESP; size值为sizeof(ts_call_reject_resp_t).
Return value	成功返回0，失败返回其它值

3.1.7. aic_srv_tele_call_volte_enable

Prototype	int32_t aic_srv_tele_call_volte_enable(uint8_t card_id, bool enable, srv_func_callback _response_callback);
Function	打开/关闭VoLTE的接口
Constraint	
Parameters	@card_id: [In]sim卡的id，0-卡槽1，1-卡槽2 @enable: [In]true-打开VoLTE，false-关闭VoLTE @_response_callback: [In] client调用该接口时传入的响应回调函数，在at命令的response结束后被调用. 响应的事件id是TS_EVENT_CALL_VOLTE_ENABLE_RESP，携带ts_call_volte_enable_resp_t数据信息. 其中，p_param可强转为ts_event_call_t指针，对应的事件id是TS_EVENT_CALL_VOLTE_ENABLE_RESP; size值为sizeof(ts_call_volte_enable_resp_t).
Return value	成功返回0，失败返回其它值

3.1.8. aic_srv_tele_call_volte_is_enable

Prototype	bool aic_srv_tele_call_volte_is_enable(uint8_t card_id);
Function	获取VoLTE是否打开的状态接口
Constraint	
Parameters	@card_id: [In]sim卡的id，0-卡槽1，1-卡槽2
Return value	true-使能VoLTE，false-未使能VoLTE

3.1.9. aic_srv_tele_call_is_emergency_number

AICXTEK Confidential

Prototype	bool aic_srv_tele_call_is_emergency_number(uint8_t card_id, char *p_number);
Function	判断号码是否为紧急号码的接口
Constraint	
Parameters	@card_id: [In]sim卡的id，0-卡槽1，1-卡槽2 @p_number: [In]号码
Return value	若是紧急号码，则返回true；若不是紧急号码，则返回false

3.1.10. aic_srv_tele_call_send_dtmf

Prototype	int32_t aic_srv_tele_call_send_dtmf(uint8_t card_id, ts_call_send_dtmf_info_t *p_dtmf_info, srv_func_callback _response_callback);
Function	保持通话的接口
Constraint	
Parameters	@card_id: [In]sim卡的id，0-卡槽1，1-卡槽2 @p_dtmf_info: [In]dtmf信息 @_response_callback: [In] client调用该接口时传入的响应回调函数，在at命令的response结束后被调用 响应的事件id是TS_EVENT_CALL_SEND_DTMF_RESP，携带ts_call_send_dtmf_resp_t数据信息. 其中，p_param可强转为ts_event_call_t指针，对应的事件id是TS_EVENT_CALL_SEND_DTMF_RESP; size值为sizeof(ts_call_send_dtmf_resp_t).
Return value	成功返回0，失败返回其它值

3.1.11. aic_srv_tele_call_have_volte_reg

Prototype	bool aic_srv_tele_call_have_volte_reg(uint8_t card_id);
Function	获取VoLTE是否注册的接口
Constraint	
Parameters	@card_id: [In]sim卡的id，0-卡槽1，1-卡槽2
Return value	true-VoLTE已注册，false-VoLTE未注册

3.1.12. aic_srv_tele_call_set_auto_answer

Prototype	int32_t aic_srv_tele_call_set_auto_answer(bool is_auto);
Function	设置自动接听的接口
Constraint	
Parameters	@is_auto: [In]true-打开自动接听，false-关闭自动接听

AICXTEK Confidential

Return value	成功返回0，失败返回其它值

3.1.13. aic_srv_tele_call_get_auto_answer

Prototype	bool aic_srv_tele_call_get_auto_answer(void);
Function	获取自动接听flag的接口
Constraint	
Parameters	无
Return value	true-自动接听打开，false-自动接听关闭

3.1.14. aic_srv_tele_call_get_call_info_list

Prototype	int32_t aic_srv_tele_call_get_call_info_list(uint8_t card_id, ts_call_info_list_t *p_call_info_list);
Function	获取通话信息列表的接口
Constraint	/* The call information for single call */ typedef struct { uint8_t card_id; /* The call index */ uint8_t call_idx; /* The telephone number */ char phone_number[TS_CALL_PHONE_NUMBER_LEN_MAX + 1]; /* The call direction: 0-MO, 1-MT */ uint8_t direction; /* The call status */ ts_call_status_e call_status; }ts_call_info_t; typedef struct { /* The number of call */ uint32_t call_number; /* The call information */ ts_call_info_t call_info[TS_CALL_NUMBER_MAX]; }ts_call_info_list_t;
Parameters	@card_id: [In]sim卡的id，0-卡槽1，1-卡槽2 @p_call_info_list: [Out]通话信息列表
Return value	成功返回0，失败返回其它值

AICXTEK Confidential

3.2. Radio

简要描述

Tele Radio Service模块主要提供了与Radio相关的功能接口，包括基础的开关radio、获取网络注册状态、信号信息、小区信息、邻近小区信息、搜网以及设置及获取imei等相关功能。

Header file	Location
aic_srv_tele_radio.h	

3.2.1. aic_srv_tele_radio_register

Prototype	int32_t aic_srv_tele_radio_register(aic_srv_bus_info_t *p_bus_info, srv_func_callback _srv_callback);
Function	tele radio service的注册接口，注册的callback用于接收service层上报的事件
Constraint	typedef struct { /* client的名称 */ char *p_client_name; /* TODO: client关注的信号相关数据信息 */ }aic_srv_bus_info_t; typedef int32_t (*srv_func_callback)(void *p_param, uint32_t size);
Parameters	@p_bus_info: [In]需要client传给service层当前client的名称以及client需要关注的信号相关数据信息 @_srv_callback: [In]client传给service层的回调函数 该回调函数中的参数p_param上报数据结构体指针，参数size表示数据的大小（包括event id） 目前可上报的事件有： 1、 TS_EVENT_RADIO_SIGNAL_INFO_UPDATED（信号强度发生变化的事件），携带ts_radio_signal_info_updated_t数据信息； 其中，p_param可强转为ts_event_radio_t指针，对应的事件id是TS_EVENT_RADIO_SIGNAL_INFO_UPDATED; size值为sizeof(ts_radio_signal_info_updated_t)； 2、 TS_EVENT_RADIO_STATUS_UPDATED（radio状态发生变化的事件），携带ts_radio_status_updated_t数据信息； 其中，p_param可强转为ts_event_radio_t指针，对应的事件id是TS_EVENT_RADIO_STATUS_UPDATED; size值为sizeof(ts_radio_status_updated_t)； 3、 TS_EVENT_RADIO_ON_RESP（radio打开或关闭的响应事件），携带ts_radio_on_resp_t数据信息. 其中，p_param可强转为ts_event_radio_t指针，对应的事件id是TS_EVENT_RADIO_ON_RESP; size值为sizeof(ts_radio_on_resp_t). 4、 TS_EVENT_RADIO_REG_STATUS_UPDATED（radio注册状态更新的事件），携带ts_radio_reg_status_updated_t数据信息. 其中，p_param可强转为ts_event_radio_t指针，对应的事件id是TS_EVENT_RADIO_REG_STATUS_UPDATED; size值为sizeof(ts_radio_reg_status_updated_t). 5、 TS_EVENT_RADIO_IMS_REG_STATUS_UPDATED（ims注册状态更新的事件），携带ts_radio_ims_reg_status_updated_t数据信息. 其中，p_param可强转为ts_event_radio_t指针，对应的事件id是TS_EVENT_RADIO_IMS_REG_STATUS_UPDATED; size值为sizeof(ts_radio_ims_reg_status_updated _t). 6、 TS_EVENT_RADIO_OPERATOR_INFO_UPDATED（运营商信息更新的事件），携带

AICXTEK Confidential

	ts_radio_operator_info_updated_t数据信息. 其中，p_param可强转为ts_event_radio_t指针，对应的事件id是TS_EVENT_RADIO_OPERATOR_INFO_UPDATED; size值为sizeof(ts_radio_operator_info_updated_t). 7、 TS_EVENT_RADIO_GET_CELL_INFO_RESP（获取小区信息的响应事件），携带ts_radio_get_cell_info_resp_t数据信息. 其中，p_param可强转为ts_event_radio_t指针，对应的事件id是TS_EVENT_RADIO_GET_CELL_INFO_RESP; size值为sizeof(ts_radio_get_cell_info_t). 8、 TS_EVENT_RADIO_GET_NEIGHBOR_CELL_INFO_LIST_RESP（获取临近小区信息的响应事件），携带ts_radio_get_neighbor_cell_info_list_resp_t数据信息. 其中，p_param可强转为ts_event_radio_t指针，对应的事件id是TS_EVENT_RADIO_GET_NEIGHBOR_CELL_INFO_LIST_RESP; size值为sizeof(ts_radio_get_neighbor_cell_info_lis_resp_t). 9、 TS_EVENT_RADIO_SET_FILTER_RESP（设置过滤条件的响应事件），携带ts_radio_set_filter_resp_t数据信息. 其中，p_param可强转为ts_event_radio_t指针，对应的事件id是TS_EVENT_RADIO_SET_FILTER_RESP; size值为sizeof(ts_radio_set_filter_resp_t). 10、 TS_EVENT_RADIO_SET_IMEI_RESP（设置IMEI的响应事件），携带ts_radio_set_imei_resp_t数据信息. 其中，p_param可强转为ts_event_radio_t指针，对应的事件id是TS_EVENT_RADIO_SET_IMEI_RESP; size值为sizeof(ts_radio_set_imei_resp_t). 11、 TS_EVENT_RADIO_SEARCH_START_RESP（网络搜索的响应事件），携带ts_radio_search_start_resp_t数据信息. 其中，p_param可强转为ts_event_radio_t指针，对应的事件id是TS_EVENT_RADIO_SEARCH_START_RESP; size值为sizeof(ts_radio_search_start_resp_t). TODO: 后续在这里补充callback中返回event id的列表.
Return value	返回给client对应的client handle，其中0代表出错

3.2.2. aic_srv_tele_radio_unregister

Prototype	int32_t aic_srv_tele_radio_unregister(int32_t client_handle);
Function	取消注册tele radio service的接口
Constraint	
Parameters	@client_handle: [In]调用aic_srv_tele_radio_register接口时返回给client的client handle值
Return value	成功返回0，失败返回其它值

3.2.3. aic_srv_tele_radio_get_singnal_info

Prototype	int32_t aic_srv_tele_radio_get_singnal_info(uint8_t card_id, ts_radio_signal_info_t *p_signal_info);
Function	获取信号强度等相关信息的接口

AICXTEK Confidential

AICXTEK Confidential

AICXTEK Confidential

AICXTEK Confidential

	TS_RADIO_NETWORK_TYPE_UNKNOWN， /* GSM */ TS_RADIO_NETWORK_TYPE_GSM, /* CDMA */ TS_RADIO_NETWORK_TYPE_CDMA, /* WCDMA */ TS_RADIO_NETWORK_TYPE_WCDMA, /* TDSCDMA */ TS_RADIO_NETWORK_TYPE_TDSCDMA, /* LTE */ TS_RADIO_NETWORK_TYPE_LTE， /* 5G NR */ TS_RADIO_NETWORK_TYPE_NR, TS_RADIO_NETWORK_TYPE_MAX }ts_radio_network_type_e; typedef struct { /* radio信号类型 */ ts_radio_network_type_e signal_type; /* 信号等级，当前支持0~4 */ uint32_t signal_level; /* 信号值（rssi） */ uint32_t signal_level_val; /* 信号强度dB */ int32_t signal_strength; }ts_radio_signal_info_t;
Parameters	@card_id: [In]sim卡的id，0-卡槽1，1-卡槽2 @p_signal_info: [Out]信号信息结构体
Return value	成功返回0，失败返回其它值.

AICXTEK Confidential

3.2.4. aic_srv_tele_radio_get_operator_info

Prototype	int32_t aic_srv_tele_radio_get_operator_info(uint8_t card_id, ts_radio_operator_info_t *p_operator_info);
Function	获取指定卡槽的运营商信息的接口
Constraint	#define TS_RADIO_OPERATOR_SHORT_NAME_LEN_MAX (10) #define TS_RADIO_OPERATOR_LONG_NAME_LEN_MAX (32) #define TS_RADIO_PLMN_MCC_LEN_MAX (3) #define TS_RADIO_PLMN_MNC_LEN_MAX (3) typedef enum { /* 无效运营商，比如无卡、无服务等 */ TS_RADIO_OPERATOR_INVALID, TS_RADIO_OPERATOR_CMCC, TS_RADIO_OPERATOR_CUCC, TS_RADIO_OPERATOR_CTCC, /* 除CMCC, CUCC和CTCC外的未知运营商 */ TS_RADIO_OPERATOR_UNKNOWN, TS_RADIO_OPERATOR_MAX }ts_radio_operator_e; typedef enum { TS_RADIO_OPERATOR_STATUS_UNKNOW = 0, TS_RADIO_OPERATOR_STATUS_AVAILABLE, TS_RADIO_OPERATOR_STATUS_CURRENT, TS_RADIO_OPERATOR_STATUS_FORBIDDEN, TS_RADIO_OPERATOR_STATUS_MAX }ts_radio_operator_status_e; typedef struct { char operator_short_name[TS_RADIO_OPERATOR_SHORT_NAME_LEN_MAX + 1]; char operator_long_name[TS_RADIO_OPERATOR_LONG_NAME_LEN_MAX + 1]; char plmn_mcc[TS_RADIO_PLMN_MCC_LEN_MAX + 1]; char plmn_mnc[TS_RADIO_PLMN_MNC_LEN_MAX + 1]; ts_radio_operator_e operator; ts_radio_operator_network_type_e network_type; ts_radio_operator_status_e status; }ts_radio_operator_info_t;
Parameters	@card_id: [In]sim卡的id，0-卡槽1，1-卡槽2 @p_operator_info: [Out]获取到的运营商信息

AICXTEK Confidential

Return value	成功返回0，失败返回其它值

3.2.5. aic_srv_tele_radio_on

Prototype	int32_t aic_srv_tele_radio_on(uint8_t card_id, bool radio_on, srv_func_callback _response_callback);
Function	开关指定卡槽位Radio的 接口
Constraint	N/A
Parameters	@card_id: [In]sim卡的id，0-卡槽1，1-卡槽2 @radio_on: [In]true-打开radio，false-关闭radio @_response_callback: [In] client调用该接口时传入的响应回调函数，在at命令的response结束后被调用. 当radio开关成功后会有TS_EVENT_RADIO_STATUS_UPDATED事件上报，并上报给client. 响应的事件id是TS_EVENT_RADIO_ON_RESP，携带ts_radio_on_resp_t数据信息. 其中，p_param可强转为ts_event_radio_t指针，对应的事件id是TS_EVENT_RADIO_ON_RESP; size值为sizeof(ts_radio_on_resp_t).
Return value	成功返回0，失败返回其它值

3.2.6. aic_srv_tele_radio_is_on

Prototype	bool aic_srv_tele_radio_is_on(uint8_t card_id);
Function	获取指定卡槽位Radio是否打开的接口
Constraint	
Parameters	@card_id: [In]sim卡的id，0-卡槽1，1-卡槽2
Return value	true-radio打开，false-radio关闭

3.2.7. aic_srv_tele_radio_get_reg_status

Prototype	int32_t aic_srv_tele_radio_get_reg_status(uint8_t card_id, ts_radio_reg_status_e *p_reg_status);
Function	获取radio注册状态的接口
Constraint	typedef enum { TS_RADIO_REG_STATUS_NO_SERVICE, TS_RADIO_REG_STATUS_IN_SERVICE,

AICXTEK Confidential

	TS_RADIO_REG_STATUS_EMERGENCY_CALL_ONLY, TS_RADIO_REG_STATUS_POWER_OFF }ts_radio_reg_status_e;
Parameters	@card_id: [In]sim卡的id，0-卡槽1，1-卡槽2 @ p_reg_status: [Out]radio的注册状态
Return value	成功返回0，失败返回其它值

3.2.8. aic_srv_tele_radio_have_ims_reg

Prototype	bool aic_srv_tele_radio_have_ims_reg(uint8_t card_id);
Function	获取radio ims是否注册的接口
Constraint	
Parameters	@card_id: [In]sim卡的id，0-卡槽1，1-卡槽2
Return value	true-radio ims注册成功，false-radio ims未注册

3.2.9. aic_srv_tele_radio_set_imei

Prototype	int32_t aic_srv_tele_radio_set_imei(uint8_t card_id, char *p_imei, srv_func_callback _response_callback);
Function	设置imei的接口
Constraint	
Parameters	@card_id: [In]sim卡的id，0-卡槽1，1-卡槽2 @p_imei: [In]设置的imei字符串 @_response_callback: [In] client调用该接口时传入的响应回调函数，在at命令的response结束后被调用. 响应的事件id是TS_EVENT_RADIO_SET_IMEI_RESP，携带ts_radio_set_imei_resp_t数据信息. 其中，p_param可强转为ts_event_radio_t指针，对应的事件id是TS_EVENT_RADIO_SET_IMEI_RESP; size值为sizeof(ts_radio_set_imei_resp_t).
Return value	成功返回0，失败返回其它值

3.2.10. aic_srv_tele_radio_get_imei

Prototype	int32_t aic_srv_tele_radio_get_imei(uint8_t card_id, char *p_imei);
Function	获取imei的接口
Constraint	
Parameters	@card_id: [In]sim卡的id，0-卡槽1，1-卡槽2

AICXTEK Confidential

	@ p_imei: [Out]imei字符串
Return value	成功返回0，失败返回其它值

3.2.11. aic_srv_tele_radio_have_cali

Prototype	bool aic_srv_tele_radio_have_cali(void);
Function	获取是否已校准的接口
Constraint	
Parameters	N/A
Return value	true-已校准，false-未校准

3.2.12. aic_srv_tele_radio_get_cell_info

Prototype	int32_t aic_srv_tele_radio_get_cell_info(uint8_t card_id, srv_func_callback _response_callback);
Function	获取小区信息的接口
Constraint	
Parameters	@card_id: [In]sim卡的id，0-卡槽1，1-卡槽2 @_response_callback: [In] client调用该接口时传入的响应回调函数，在at命令的response结束后被调用. 响应的事件id是TS_EVENT_RADIO_GET_CELL_INFO_RESP，携带ts_radio_get_cell_info_resp_t数据信息. 其中，p_param可强转为ts_event_radio_t指针，对应的事件id是TS_EVENT_RADIO_GET_CELL_INFO_RESP; size值为sizeof(ts_radio_get_cell_info_resp_t).
Return value	成功返回0，失败返回其它值

3.2.13. aic_srv_tele_radio_get_neighbor_cell_info_list

Prototype	int32_t aic_srv_tele_radio_get_neighbor_cell_info_list(uint8_t card_id, srv_func_callback _response_callback);
Function	获取邻近小区信息的接口
Constraint	
Parameters	@card_id: [In]sim卡的id，0-卡槽1，1-卡槽2 @_response_callback: [In] client调用该接口时传入的响应回调函数，在at命令的response结束后被调用.

AICXTEK Confidential

	响应的事件id是TS_EVENT_RADIO_GET_NEIGHBOR_CELL_INFO_LIST_RESP，携带ts_radio_get_neighbor_cell_info_list_resp_t数据信息. 其中，p_param可强转为ts_event_radio_t指针，对应的事件id是TS_EVENT_RADIO_GET_NEIGHBOR_CELL_INFO_LIST_RESP; size值为sizeof(ts_radio_get_neighbor_cell_info_list_resp_t).
Return value	成功返回0，失败返回其它值

3.2.14. aic_srv_tele_radio_set_filter

Prototype	int32_t aic_srv_tele_radio_set_filter(uint8_t card_id, uint32_t filter_val, srv_func_callback _response_callback);
Function	设置过滤上报信息的接口
Constraint	
Parameters	@card_id: [In]sim卡的id，0-卡槽1，1-卡槽2 @ filter_val: [In]过滤值 #define TS_RADIO_FILTER_ALL (0xFFFFFFFF) #define TS_RADIO_FILTER_NONE (0) #define TS_RADIO_FILTER_REG_STATUS (1 << 0) #define TS_RADIO_FILTER_SIGNAL_INFO (1 << 1) @_response_callback: [In] client调用该接口时传入的响应回调函数，在at命令的response结束后被调用. 响应的事件id是TS_EVENT_RADIO_SET_FILTER_RESP，携带ts_radio_set_filter_resp_t数据信息. 其中，p_param可强转为ts_event_radio_t指针，对应的事件id是TS_EVENT_RADIO_SET_FILTER_RESP; size值为sizeof(ts_radio_set_filter_resp_t).
Return value	成功返回0，失败返回其它值

3.2.15. aic_srv_tele_radio_start_search

Prototype	int32_t aic_srv_tele_radio_start_search(uint8_t card_id, srv_func_callback _response_callback);
Function	搜索网络的接口
Constraint	
Parameters	@card_id: [In]sim卡的id，0-卡槽1，1-卡槽2

AICXTEK Confidential

	@_response_callback: [In] client调用该接口时传入的响应回调函数，在at命令的response结束后被调用. 响应的事件id是TS_EVENT_RADIO_START_SEARCH_RESP，携带ts_radio_start_search_resp_t数据信息. 其中，p_param可强转为ts_event_radio_t指针，对应的事件id是TS_EVENT_RADIO_START_SEARCH_RESP; size值为sizeof(ts_radio_start_search_resp_t).
Return value	成功返回0，失败返回其它值

3.3. SIM

简要描述

Tele Sim Service模块主要提供了与Sim卡相关的功能接口，包括基础的获取sim卡状态、plmn、imsi、iccid、msisdn等信息的相关功能。

Header file	Location
aic_srv_tele_sim.h	

3.3.1. aic_srv_tele_sim_register

Prototype	int32_t aic_srv_tele_sim_register(aic_srv_bus_info_t *p_bus_info, srv_func_callback _srv_callback)
Function	tele sim service的注册接口，注册的callback用于接收service层上报的事件
Constraint	typedef struct { /* client的名称 */ char *p_client_name; /* TODO: client关注的信号相关数据信息 */ }aic_srv_bus_info_t; typedef int32_t (*srv_func_callback)(void *p_param, uint32_t size);
Parameters	@p_bus_info: [In]需要client传给service层当前client的名称以及client需要关注的信号相关数据信息 @_srv_callback: [In]client传给service层的回调函数 该回调函数中的参数p_param上报数据结构体指针，参数size表示数据的大小（包括event id） 目前可上报的事件有： 1、 TS_EVENT_SIM_STATUS_UPDATED（sim卡状态发生变化的事件），不携带数据； 其中，p_param可强转为ts_event_sim_t指针，对应的事件id是TS_EVENT_SIM_STATUS_UPDATED; size值为4； 2、 TS_EVENT_SIM_STATUS_UPDATED_WITH_DATA（sim卡状态发生变化的事件），携带ts_sim_status_updated_with_data_t数据信息； 其中，p_param可强转为ts_event_sim_t指针，对应的事件id是TS_EVENT_SIM_STATUS_UPDATED_WITH_DATA; size值为sizeof(ts_sim_status_updated_with_data_t)； 3、 TS_EVENT_SIM_SET_PIN_LOCK_STATUS_RESP（设置sim卡pin锁状态的响应事件），携带ts_sim_set_pin_lock_status_resp_t数据信息；

AICXTEK Confidential

	其中，p_param可强转为ts_event_sim_t指针，对应的事件id是TS_EVENT_SIM_SET_PIN_LOCK_STATUS_RESP; size值为sizeof(ts_sim_set_pin_lock_status_resp_t)； 4、 TS_EVENT_SIM_GET_PIN_LOCK_STATUS_RESP（获取sim卡pin锁状态的响应事件），携带ts_sim_get_pin_lock_status_resp_t数据信息； 其中，p_param可强转为ts_event_sim_t指针，对应的事件id是TS_EVENT_SIM_GET_PIN_LOCK_STATUS_RESP; size值为sizeof(ts_sim_get_pin_lock_status_resp_t)； 5、 TS_EVENT_SIM_UNLOCK_PIN_RESP（解锁pin码的响应事件），携带ts_sim_unlock_pin_resp_t数据信息； 其中，p_param可强转为ts_event_sim_t指针，对应的事件id是TS_EVENT_SIM_UNLOCK_PIN_RESP; size值为sizeof(ts_sim_unlock_pin_resp_t)； 6、 TS_EVENT_SIM_CHANGE_PIN_RESP（修改pin码的响应事件），携带ts_sim_change_pin_resp_t数据信息； 其中，p_param可强转为ts_event_sim_t指针，对应的事件id是TS_EVENT_SIM_CHANGE_PIN_RESP; size值为sizeof(ts_sim_change_pin_resp_t)； 7、 TS_EVENT_SIM_GET_PIN_REMAIN_TIMES_RESP（获取pin码剩余次数的响应事件），携带ts_sim_get_pin_remain_times_resp_t数据信息； 其中，p_param可强转为ts_event_sim_t指针，对应的事件id是TS_EVENT_SIM_GET_PIN_REMAIN_TIMES_RESP; size值为sizeof(ts_sim_get_pin_remain_times_resp_t)； 8、 TS_EVENT_SIM_SELECT_CARD_RESP（选卡的响应事件），携带ts_sim_select_card_resp_t数据信息； 其中，p_param可强转为ts_event_sim_t指针，对应的事件id是TS_EVENT_SIM_SELECT_CARD_RESP; size值为sizeof(ts_sim_select_card_resp_t)； TODO: 后续在这里补充callback中返回event id的列表.
Return value	返回给client对应的client handle，其中0代表出错

3.3.2. aic_srv_tele_sim_unregister

Prototype	int32_t aic_srv_tele_sim_unregister(int32_t client_handle)
Function	tele sim service的去注册接口
Constraint	
Parameters	@client_handle: [In]调用aic_srv_tele_sim_register接口时返回给client的client handle值
Return value	成功返回0，失败返回其它值

3.3.3. aic_srv_tele_sim_get_status

Prototype	int32_t aic_srv_tele_sim_get_status(uint8_t card_id, ts_sim_status_e *p_sim_status)
Function	获取sim卡状态的接口
Constraint	typedef enum

AICXTEK Confidential

	{ /* sim卡not ready*/ TS_SIM_STATUS_NOT_READY = -1, /* sim卡不存在 */ TS_SIM_STATUS_NOT_PRESENT, /* sim卡ready */ TS_SIM_STATUS_READY, /* sim卡pin */ TS_SIM_STATUS_PIN, /* sim卡puk */ TS_SIM_STATUS_PUK, /* sim卡pin2 */ TS_SIM_STATUS_PIN2, /* sim卡puk2 */ TS_SIM_STATUS_PUK2， TS_SIM_STATUS_MAX }ts_sim_status_e;
Parameters	@card_id: [In]sim卡的id，0-卡槽1，1-卡槽2 @p_sim_status: [Out]获取到的sim卡状态
Return value	成功返回0，失败返回其它值

3.3.4. aic_srv_tele_sim_set_pin_lock_status

Prototype	int32_t aic_srv_tele_sim_set_pin_lock_status(uint8_t card_id, ts_sim_pin_lock_status_t *p_lock_status, srv_func_callback _response_callback)
Function	设置sim卡pin码锁状态的接口
Constraint	#define TS_SIM_PIN_PWD_LEN_MAX 8 typedef struct { bool is_enable;

AICXTEK Confidential

	char pin_pwd[TS_SIM_PIN_PWD_LEN_MAX]; }ts_sim_pin_lock_status_t;
Parameters	@card_id: [In]sim卡的id，0-卡槽1，1-卡槽2 @p_lock_status: [In]sim卡pin码锁状态指针 @_response_callback: [In] client调用该接口时传入的响应回调函数，在at命令的response结束后被调用. 响应的事件id是TS_EVENT_SIM_SET_PIN_LOCK_STATUS_RESP，携带ts_sim_set_pin_lock_status_resp_t数据信息. 其中，p_param可强转为ts_event_radio_t指针，对应的事件id是TS_EVENT_SIM_SET_PIN_LOCK_STATUS_RESP; size值为sizeof(ts_sim_set_pin_lock_status_resp_t).
Return value	成功返回0，失败返回其它值

3.3.5. aic_srv_tele_sim_get_pin_lock_status

Prototype	int32_t aic_srv_tele_sim_get_pin_lock_status(uint8_t card_id, srv_func_callback _response_callback)
Function	获取sim卡pin码锁状态的接口
Constraint	
Parameters	@card_id: [In]sim卡的id，0-卡槽1，1-卡槽2 @_response_callback: [In] client调用该接口时传入的响应回调函数，在at命令的response结束后被调用. 响应的事件id是TS_EVENT_SIM_GET_PIN_LOCK_STATUS_RESP，携带ts_sim_get_pin_lock_status_resp_t数据信息. 其中，p_param可强转为ts_event_radio_t指针，对应的事件id是TS_EVENT_SIM_GET_PIN_LOCK_STATUS_RESP; size值为sizeof(ts_sim_get_pin_lock_status_resp_t).
Return value	成功返回0，失败返回其它值

3.3.6. aic_srv_tele_sim_unlock_pin

Prototype	int32_t aic_srv_tele_sim_unlock_pin(uint8_t card_id, char *p_pin,

AICXTEK Confidential

	srv_func_callback _response_callback)
Function	解锁sim卡pin码的接口
Constraint	
Parameters	@card_id: [In]sim卡的id，0-卡槽1，1-卡槽2 @p_pin: [In]sim卡的pin码 @_response_callback: [In] client调用该接口时传入的响应回调函数，在at命令的response结束后被调用. 响应的事件id是TS_EVENT_SIM_UNLOCK_PIN_RESP，携带ts_sim_unlock_pin_resp_t数据信息. 其中，p_param可强转为ts_event_radio_t指针，对应的事件id是TS_EVENT_SIM_UNLOCK_PIN_RESP; size值为sizeof(ts_sim_unlock_pin_resp_t).
Return value	成功返回0，失败返回其它值

3.3.7. aic_srv_tele_sim_change_pin

Prototype	int32_t aic_srv_tele_sim_change_pin(uint8_t card_id, char *p_new_pin, char *p_old_pin, srv_func_callback _response_callback)
Function	修改sim卡pin码的接口
Constraint	
Parameters	@card_id: [In]sim卡的id，0-卡槽1，1-卡槽2 @p_new_pin: [In]sim卡的新pin码 @p_old_pin: [In]sim卡的旧pin码 @_response_callback: [In] client调用该接口时传入的响应回调函数，在at命令的response结束后被调用. 响应的事件id是TS_EVENT_SIM_CHANGE_PIN_RESP，携带ts_sim_change_pin_resp_t数据信息. 其中，p_param可强转为ts_event_radio_t指针，对应的事件id是TS_EVENT_SIM_CHANGE_PIN_RESP; size值为sizeof(ts_sim_change_pin_resp_t).
Return value	成功返回0，失败返回其它值

AICXTEK Confidential

3.3.8. aic_srv_tele_sim_get_pin_remain_times

Prototype	int32_t aic_srv_tele_sim_get_pin_remain_times(uint8_t card_id, srv_func_callback _response_callback)
Function	获取sim卡pin码剩余次数的接口
Constraint	
Parameters	@card_id: [In]sim卡的id，0-卡槽1，1-卡槽2 @_response_callback: [In] client调用该接口时传入的响应回调函数，在at命令的response结束后被调用. 响应的事件id是TS_EVENT_SIM_GET_PIN_REMAIN_TIMES_RESP，携带ts_sim_get_pin_remain_times_resp_t数据信息. 其中，p_param可强转为ts_event_radio_t指针，对应的事件id是TS_EVENT_SIM_GET_PIN_REMAIN_TIMES_RESP; size值为sizeof(ts_sim_get_pin_remain_times_resp_t).
Return value	成功返回0，失败返回其它值

3.3.9. aic_srv_tele_sim_select_card

Prototype	int32_t aic_srv_tele_sim_select_card(uint8_t card_id, srv_func_callback _response_callback)
Function	选卡的接口
Constraint	
Parameters	@card_id: [In]sim卡的id，0-卡槽1，1-卡槽2 @_response_callback: [In] client调用该接口时传入的响应回调函数，在at命令的response结束后被调用. 响应的事件id是TS_EVENT_SIM_SELECT_CARD_RESP，携带ts_sim_select_card_resp_t数据信息. 其中，p_param可强转为ts_event_radio_t指针，对应的事件id是TS_EVENT_SIM_SELECT_CARD_RESP; size值为sizeof(ts_sim_select_card_resp_t).
Return value	成功返回0，失败返回其它值

3.3.10. aic_ srv_tele_sim_get_current_card

AICXTEK Confidential

Prototype	int32_t aic_srv_tele_sim_get_current_card (uint8_t *p_card_id)
Function	获取sim卡存在状态的接口
Constraint	
Parameters	@p_card_id: [Out]获取到的当前sim卡的card id
Return value	成功返回0，失败返回其它值

3.3.11. aic_ srv_tele_sim_get_exist_status

Prototype	int32_t aic_srv_tele_sim_get_exist_status (uint32_t *p_sim_status)
Function	获取sim卡存在状态的接口
Constraint	带有_sync后缀的api代表该接口是耗时接口
Parameters	@p_sim_status: [Out]获取到的sim卡存在状态的值，0：未插入sim卡，1：sim1存在，2：sim2存在，3：sim1和sim2都存在
Return value	成功返回0，失败返回其它值

3.3.12. aic_srv_tele_sim_get_imsi

Prototype	int32_t aic_srv_tele_sim_get_imsi(uint8_t card_id, char *p_sim_imsi)
Function	获取sim卡imsi字符串的接口
Constraint	
Parameters	@card_id: [In]sim卡的id，0-卡槽1，1-卡槽2 @p_sim_imsi: [Out]获取到的sim卡imsi字符串
Return value	成功返回0，失败返回其它值

3.3.13. aic_ srv_tele_sim_get_msisdn

Prototype	int32_t aic_srv_tele_sim_get_msisdn(uint8_t card_id, char *p_sim_msisdn)
Function	获取sim卡msisdn本机号码的接口
Constraint	
Parameters	@card_id: [In]sim卡的id，0-卡槽1，1-卡槽2 @p_sim_msisdn: [Out]获取到的sim卡msisdn本机号码字符串
Return value	成功返回0，失败返回其它值

3.4. Data

简要描述

Tele Data Service模块主要提供了与Data相关的功能接口，包括基础的pdp激活、去激活、获取pdp信息等相关功能。

Header file	Location

AICXTEK Confidential

aic_srv_tele_data.h	

3.4.1. aic_srv_tele_data_register

Prototype	int32_t aic_srv_tele_data_register(aic_srv_bus_info_t *p_bus_info, srv_func_callback _srv_callback)
Function	tele data service的注册接口，注册的callback用于接收service层上报的事件
Constraint	typedef struct { /* client的名称 */ char *p_client_name; /* TODO: client关注的信号相关数据信息 */ }aic_srv_bus_info_t; typedef int32_t (*srv_func_callback)(void *p_param, uint32_t size);
Parameters	@p_bus_info: [In]需要client传给service层当前client的名称以及client需要关注的信号相关数据信息 @_srv_callback: [In]client传给service层的回调函数 该回调函数中的参数p_param上报数据结构体指针，参数size表示数据的大小（包括event id） 目前可上报的事件有： 1、 TS_EVENT_DATA_PDP_STATUS_UPDATED（pdp状态发生变化的事件），携带ts_data_pdp_status_updated_t数据信息； 其中，p_param可强转为ts_event_data_t指针，对应的事件id是TS_EVENT_DATA_PDP_STATUS_UPDATED; size值为sizeof(ts_data_pdp_status_updated_t)； TODO: 后续在这里补充callback中返回event id的列表
Return value	返回给client对应的client handle，其中0代表出错

3.4.2. aic_srv_tele_data_unregister

AICXTEK Confidential

Prototype	int32_t aic_srv_tele_data_unregister(int32_t client_handle)
Function	tele data service的去注册接口
Constraint	
Parameters	@client_handle: [In]调用aic_srv_tele_data_register接口时返回给client的client handle值
Return value	成功返回0，失败返回其它值

3.4.3. aic_srv_tele_data_get_internet_pdp_info

Prototype	int32_t aic_srv_tele_data_get_internet_pdp_info(uint8_t card_id, ts_data_pdp_info_t *p_pdp_data_info)
Function	获取internet这路pdp信息的接口
Constraint	
Parameters	@card_id: [In]sim卡的id，0-卡槽1，1-卡槽2 @p_pdp_data_info: [Out]internet这路pdp的信息
Return value	成功返回0，失败返回其它值

3.5. SMS

简要描述

Tele SMS Service模块主要提供了与SMS(短信)相关的功能接口，包括

1. 发送短信
2. 删除SIM卡中的短信
3. 设置/读取短信中心号码
4. 获取SIM卡中短信的index列表
5. 读取SIM卡中的短信
6. 设置是否接收短信状态报告的通知

Header file	Location
aic_srv_tele_sms.h	

3.5.1. aic_srv_tele_sms_register

Prototype	int32_t aic_srv_tele_sms_register(aic_srv_bus_info_t *p_bus_info, srv_func_callback _srv_callback)
Function	tele sms service的注册接口，注册的callback用于接收service层上报的事件
Constraint	typedef struct { /* client的名称 */ char *p_client_name;

AICXTEK Confidential

	/* TODO: client关注的信号相关数据信息 */ }aic_srv_bus_info_t; typedef int32_t (*srv_func_callback)(void *p_param, uint32_t size);
Parameters	@p_bus_info: [IN] client information. Reserved for future use @_srv_callback: [IN] client callback 该回调函数中的参数p_param上报数据结构体指针，参数size表示数据的大小（包括event id） 目前可上报的事件有: 1、 TS_EVENT_SMS_SEND_RESP 2、 TS_EVENT_SMS_DEL_IN_SIM_RESP 3、 TS_EVENT_SMS_SET_SMSC_RESP 4、 TS_EVENT_SMS_GET_SMSC_RESP 5、 TS_EVENT_SMS_GET_LIST_IN_SIM_RESP 6、 TS_EVENT_SMS_READ_IN_SIM_RESP 7、 TS_EVENT_SMS_NEW_IND 8、 TS_EVENT_SMS_STATUS_REPORT_IND 9、 TS_EVENT_SMS_STORAGE_STATUS_IND
Return value	返回给client对应的client handle，其中0代表出错

3.5.2. aic_srv_tele_sms_unregister

Prototype	int32_t aic_srv_tele_sms_unregister(int32_t client_handle)
Function	tele sms service的去注册接口
Constraint	
Parameters	@client_handle: [In]调用aic_srv_tele_sms_register接口时返回给client的client handle值
Return value	成功返回0，失败返回-1

3.5.3. aic_srv_tele_sms_send

Prototype	int32_t aic_srv_tele_sms_send(uint8_t card_id, const ts_sms_send_info_t *p_sms_info, srv_func_callback _response_callback)
Function	发送短信
Constraint	/** * struct ts_sms_send_info_t - info of sms sending * @p_smsc: sms center, use "" or NULL to set smsc as default one

AICXTEK Confidential

	* @p_number: recipient number * @p_text: sms text to send, encoded as UTF8 * @is_write_to_sim: set true to write the sms to SIM */ typedef struct { const char *p_smsc; const char *p_number; const char *p_text; bool is_write_to_sim; } ts_sms_send_info_t; /** * struct ts_sms_general_resp_t - general response of sms operation * @event_id: event id, refer to ts_event_sms_e * @card_id: sim slot * @err_code: error code. 0 for success, otherwise refers to HRilErrNumber */ typedef struct { ts_event_sms_e event_id; uint8_t card_id; int32_t err_code; } ts_sms_general_resp_t; /** * struct ts_sms_send_resp_t - response of sms sending * @base_info: basic response info * @index: index of the pdu written into SIM. -1 means not saved. */ typedef struct { ts_sms_general_resp_t base_info; int32_t index; } ts_sms_send_resp_t;
Parameters	@card_id: [IN] sim slot. Reserved for future use @p_sms_info: [IN] pointer to sms info @_response_callback: [IN] additional cb. reserved for future use
Return value	成功返回0，失败返回-1 此接口为异步处理函数，接口执行结果如果返回0，那么后续将会收到事件：TS_EVENT_SMS_SEND_RESP 参数数据类型： ts_sms_send_resp_t base_info中的err_code，仅代表发送是否成功。

3.5.4. aic_srv_tele_sms_del_in_sim

AICXTEK Confidential

Prototype	int32_t aic_srv_tele_sms_del_in_sim(uint8_t card_id, int32_t *p_indexes, int32_t count, srv_func_callback _response_callback)
Function	从SIM卡中删除指定一条或多条短信
Constraint	/** * struct ts_sms_del_in_sim_resp_t - response of deleting in sim * @base_info: basic response info */ typedef struct { ts_sms_general_resp_t base_info; } ts_sms_del_in_sim_resp_t;
Parameters	@card_id: [IN] sim slot. Reserved for future use @p_indexes: [IN] index list of the SMS records in SIM @count: [IN] count of indexes @_response_callback: [IN] additional cb. reserved for future use
Return value	成功返回0，失败返回-1 此接口为异步处理函数，接口执行结果如果返回0，那么后续将会收到事件：TS_EVENT_SMS_DEL_IN_SIM_RESP 参数数据类型： ts_sms_del_in_sim_resp_t

3.5.5. aic_srv_tele_sms_set_smsc

Prototype	int32_t aic_srv_tele_sms_set_smsc(uint8_t card_id, const char *p_address, srv_func_callback _response_callback)
Function	设置短信中心号码
Constraint	/** * struct ts_sms_set_smsc_resp_t - response of setting smsc * @base_info: basic response info */ typedef struct { ts_sms_general_resp_t base_info; } ts_sms_set_smsc_resp_t;
Parameters	@card_id: [IN] sim slot. Reserved for future use @p_address: [IN] smsc of string type @_response_callback: [IN] additional cb. reserved for future use
Return value	成功返回0，失败返回-1 此接口为异步处理函数，接口执行结果如果返回0，那么后续将会收到事件：TS_EVENT_SMS_SET_SMSC_RESP 参数数据类型： ts_sms_set_smsc_resp_t

AICXTEK Confidential

3.5.6. aic_srv_tele_sms_get_smsc

Prototype	int32_t aic_srv_tele_sms_get_smsc(uint8_t card_id, srv_func_callback _response_callback)
Function	读取短信中心号码
Constraint	/** * struct ts_sms_get_smsc_resp_t - response of sms getting smsc * @base_info: basic response info * @smsc: smsc of string type */ typedef struct { ts_sms_general_resp_t base_info; char smsc[TS_SMS_NUM_MAX_LEN + 1]; } ts_sms_get_smsc_resp_t;
Parameters	@card_id: [IN] sim slot. Reserved for future use @_response_callback: [IN] additional cb. reserved for future use
Return value	成功返回0，失败返回-1 此接口为异步处理函数，接口执行结果如果返回0，那么后续将会收到事件：TS_EVENT_SMS_SET_SMSC_RESP 参数数据类型： ts_sms_get_smsc_resp_t

3.5.7. aic_srv_tele_sms_get_list_in_sim

Prototype	int32_t aic_srv_tele_sms_get_list_in_sim(uint8_t card_id, ms_box_type_e box, srv_func_callback _response_callback)
Function	获取短信index和状态列表
Constraint	/** * struct ts_sms_entry_t - sms record entry * @index: index * @status: status of the item */ typedef struct { int32_t index; ts_sms_rec_status_e status; } ts_sms_entry_t; /** * struct ts_sms_get_list_in_sim_resp_t - response of sms getting entry list * @base_info: basic response info * @box: box type

AICXTEK Confidential

	* @count: total count of info list
	* @list_start: place holder of list start
	* Note: the size of ts_sms_get_list_in_sim_resp_t is variable.
	* Suggestion to use:
	* ts_sms_get_list_in_sim_resp_t *resp = (ts_sms_get_list_in_sim_resp_t *)p_param;
	* ts_sms_entry_t *list = (ts_sms_entry_t *)(&resp->list_start);
	* for (uint16_t i = 0; i < resp->count; i++) {
	* // try to use list[i] as each item
	* }
	*/
	typedef struct {
	ts_sms_general_resp_t base_info;
	ts_sms_box_type_e box;
	uint16_t count;
	uint32_t list_start;
	} ts_sms_get_list_in_sim_resp_t;

Parameters	@card_id: [IN] sim slot. Reserved for future use @box: [IN] box type @_response_callback: [IN] additional cb. reserved for future use
Return value	成功返回0，失败返回-1 此接口为异步处理函数，接口执行结果如果返回0，那么后续将会收到事件：TS_EVENT_SMS_GET_LIST_IN_SIM_RESP 参数数据类型： ts_sms_get_list_in_sim_resp_t

3.5.8. aic_srv_tele_sms_read_in_sim

Prototype	int32_t aic_srv_tele_sms_read_in_sim(uint8_t card_id, uint32_t index, srv_func_callback _response_callback)
Function	读取指定短信内容
Constraint	/** * struct ts_sms_read_in_sim_resp_t - response of sms reading entry in sim * @base_info: basic response info * @index: index * @sms_class: sms class * @number: recipient(submit) / origination(deliver) number * @time_stamp: timestamp of the sms * @text_start: place holder of text start, encoded as UTF8, ended with '\0' * Note: the size of ts_sms_read_in_sim_resp_t is variable. * Suggestion to use: * ts_sms_read_in_sim_resp_t *resp = (ts_sms_read_in_sim_resp_t *)p_param;

AICXTEK Confidential

	* char *text = (char *)(&resp->text_start); */ typedef struct { ts_sms_general_resp_t base_info; int32_t index; ts_sms_class_t sms_class; char number[TS_SMS_NUM_MAX_LEN + 1]; ts_sms_time_stamp_t time_stamp; uint32_t text_start; } ts_sms_read_in_sim_resp_t;
Parameters	@card_id: [IN] sim slot. Reserved for future use @index: [IN] index of the record in SIM
Return value	成功返回0，失败返回-1 此接口为异步处理函数，接口执行结果如果返回0，那么后续将会收到事件：TS_EVENT_SMS_READ_IN_SIM_RESP 参数数据类型： ts_sms_read_in_sim_resp_t

3.5.9. aic_srv_tele_sms_enable_status_report

Prototype	int32_t aic_srv_tele_sms_enable_status_report(uint8_t card_id, bool enable)
Function	设置状态报告是否通知
Constraint	
Parameters	@card_id: [IN] sim slot. Reserved for future use @enable: [IN] true to enable, false to disable
Return value	成功返回0，失败返回-1

3.5.10. aic_srv_tele_sms_set_valid_period

Prototype	int32_t aic_srv_tele_sms_set_valid_period(uint8_t card_id, bool is_present, uint8_t tp_vp)
Function	设置有效周期
Constraint	
Parameters	* @card_id: [IN] sim slot. Reserved for future use * @is_present: [IN] tp-vp is present or not * @tp_vp: [IN] value options refer to 3GPP TS 23.040 9.2.3.12.1 TP-VP (Relative format)
Return value	成功返回0，失败返回-1

3.6. Phonebook

简要描述

AICXTEK Confidential

Tele Phonebook模块主要提供了与Phonebook相关的功能接口，包括基础的Phonebook的读取和写入等相关功能。

Header file	Location
aic_srv_tele_pb.h	

3.6.1. aic_srv_tele_pb_register

Prototype	int32_t aic_srv_tele_pb_register(aic_srv_bus_info_t *p_bus_info, srv_func_callback _srv_callback)
Function	tele pb的注册接口，注册的callback用于接收service层上报的事件
Constraint	typedef struct { /* client的名称 */ char *p_client_name; /* TODO: client关注的信号相关数据信息 */ }aic_srv_bus_info_t; typedef int32_t (*srv_func_callback)(void *p_param, uint32_t size);
Parameters	@p_bus_info: [IN] bus info. @_srv_callback: [IN] callback function client passed to servic. the callback function param: p_param for event struct point, size for struct size current events id： 1. TS_EVENT_PB_INIT_IND 2. TS_EVENT_PB_WRITE_RESP 3. TS_EVENT_PB_READ_RESP 4. TS_EVENT_PB_GET_STATUS_RESP
Return value	返回给client对应的client handle，小于0代表出错.

3.6.2. aic_srv_tele_pb_unregister

Prototype	int32_t aic_srv_tele_pb_unregister(int32_t client_handle)
Function	tele pb service的去注册接口
Constraint	
Parameters	@client_handle: [In]调用aic_srv_tele_pb_register接口时返回给client的client handle值
Return value	成功返回0，失败返回其它值

3.6.3. aic_srv_tele_pb_get_status

Prototype	int32_t aic_srv_tele_pb_get_status(uint8_t card_id, srv_func_callback _response_callback)

AICXTEK Confidential

Function	获取phonebook存储状态的接口
Constraint	typedef struct { uint16_t total; uint16_t used; uint16_t first_index; } ts_pb_status_t; typedef struct { TS_EVENT_PB_STRUCT_HEAD int32_t err; uint8_t card_id; ts_pb_status_t pb_status; } ts_pb_get_status_resp_t;
Parameters	@card_id: [In]sim卡的id，0-卡槽1，1-卡槽2，目前仅支持单卡(0) @_response_callback: [IN] callback function to notify result of this request, if set to NULL notify pass to global srv_callback.
Return value	成功返回0，失败返回其它值

3.6.4. aic_srv_tele_pb_write

Prototype	int32_t aic_srv_tele_pb_write(uint8_t card_id, ts_pb_info_t *p_contact, srv_func_callback _response_callback)
Function	发起一次phonebook写请求
Constraint	typedef struct { /* contact invalid if name[0] is '\0' */ char name[TS_PB_MAX_NAME_LEN]; char number[TS_PB_MAX_NUMBER_LEN]; uint16_t index; } ts_pb_info_t; typedef struct { TS_EVENT_PB_STRUCT_HEAD int32_t err; uint8_t card_id; uint16_t written_index; } ts_pb_write_resp_t;
Parameters	@card_id: [In]sim卡的id，0-卡槽1，1-卡槽2，目前仅支持单卡(0) @p_contact: [IN] contact info. @_response_callback: [IN] callback function to notify result of this request, if set to NULL notify pass to global srv_callback.
Return value	成功返回0，失败返回其它值

3.6.5. aic_srv_tele_pb_read

AICXTEK Confidential

Prototype	int32_t aic_srv_tele_pb_read(uint8_t card_id, uint16_t count, uint16_t first_index, srv_func_callback _response_callback)
Function	发起一次phonebook读请求
Constraint	typedef struct { /* contact invalid if name[0] is '\0' */ char name[TS_PB_MAX_NAME_LEN]; char number[TS_PB_MAX_NUMBER_LEN]; uint16_t index; } ts_pb_info_t; typedef struct { TS_EVENT_PB_STRUCT_HEAD int32_t err; uint8_t card_id; ts_pb_info_t *p_contact; uint16_t count; bool end; } ts_pb_read_resp_t;
Parameters	@card_id: [In]sim卡的id，0-卡槽1，1-卡槽2，目前仅支持单卡(0) @count: [IN] read contacts count. @first_index: [IN] first contacts index to read from. @_response_callback: [IN] callback function to notify result of this request, if set to NULL notify pass to global srv_callback.
Return value	成功返回0，失败返回其它值

AICXTEK Confidential